Chapter 15

CS2 course package - Webster University

Xiaoyuan Suo[1] and Peter Maher[1]

[1]Webster University

PDC course material used in CS2 at Webster University

Abstract

|This chapter presents a course transformation in an introductory computer science sequence (CS2) at Webster University, which is a primarily undergraduate institution, where Parallel and Distributed Computing (PDC) concepts are integrated without requiring prior background in systems or advanced mathematics. The target audience includes STEM undergraduates with little to no programming experience. The primary learning outcomes are for students to (1) understand core PDC ideas such as parallelism, speedup, and efficiency, (2) reason about workload distribution and performance trade-offs, and (3) connect conceptual understanding to simple implementations in C++. The transformation emphasizes accessible, low-barrier entry points through a combination of brief interactive visualizations, selective unplugged activities, and lightweight programming exercises, enabling students to grasp foundational performance concepts early in the curriculum.|

We designed a sequence of activities that progressively build students' understanding of PDC concepts, moving from intuitive, unplugged experiences to guided visualizations and finally to lightweight programming tasks.

Programming Language Employed: |C++|

Relevant core courses: |CS2|

Pre-requisites: |CS1|

Textbook: |Tony Gaddic|

Relevant PDC topics:

Relevant PDC topics and Blooms levels

PDC Concept	Blooms Level
Introduction to Parallel Computing	K
Task Decomposition and Partitioning	K
Speedup and Efficiency	C
Performance Measurement and Benchmarking	K

Learning outcomes:

1. |Discuss and apply good software development practices involving design, coding, and testing.|

2. |Demonstrate their understanding of recursive functions.|
3. |Describe and compare major searching and sorting algorithms.|
4. |Compare and make critical assessments of algorithms for specific applications.|
5. |Describe and use binary trees.|
6. |Demonstrate a good understanding of graphs, and their applications.|
7. |Understand the use and benefit of Parallel Programming|

Context for use: |Webster University is a primarily undergraduate institution located in St. Louis. The Department of Computer and Information Sciences offers several undergraduate programs, including a B.S. in Computer Science with multiple emphasis areas. High-level programming instruction is delivered through a two-course CS2 sequence consisting of COSC 3050 and COSC 3100. These courses introduce students to important concepts in C++ programming, including recursion, use of classes, several sorting algorithms, binary trees, heaps, AVL trees, huffman code, graphs, finding the shortest path between cities. Students develop a good understanding of important programming topics by developing several programs related to these topics. Typical enrollment in each course ranges from approximately 12 to 25 students per semester.|

15.1 Introduction

At Webster University, we have explored ways to introduce Parallel and Distributed Computing (PDC) concepts in several courses, particularly higher-level examples within a CS2 course taught in C++. For example, using PDC for handling various sorting algorithms is a clearly beneficial. Learning these types of examples clarified the benefits of PDC.

15.2 How CS2 was changed and PDC topics integrated

We modified several traditional CS2 topics, particularly searching and sorting, to incorporate introductory parallel and distributed computing (PDC) concepts without significantly changing the overall structure of the course. Loops were used to introduce ideas such as data parallelism, independent task execution, and workload decomposition by demonstrating how repetitive operations can be divided among multiple workers. Arrays provided a natural context for discussing partitioning large datasets into smaller segments that can be processed concurrently. Simple examples such as searching, counting, and grid-based simulations were adapted to help students recognize opportunities for parallel execution while still reinforcing core programming concepts.

15.2.1 Integrated Syllabi (Pre and Post)

The overall structure of the CS2 sequence remained largely unchanged, allowing the integration of parallel and distributed computing (PDC) concepts without removing core introductory programming content. Instead, selected topics were enhanced with parallel

thinking activities, demonstrations, and programming exercises. Traditional CS2 concepts such as searching, sorting, and simulations were used as natural entry points for introducing decomposition, concurrency, workload partitioning, and performance considerations.

15.2.2 Highlights

Core CS2 content remained largely unchanged, including recursion, sorting, object-oriented programming, and debugging. The primary changes involved embedding PDC concepts into existing topics through activities, simulations, and performance-oriented examples. Some highly language-specific or less essential STL-focused material was reduced or simplified to create instructional time for the new content.

Unplugged Activities

Several examples of the benefits of parallel programming are discussed in CS1 courses. Then in CS2 we are able to develop more code to perform the benefits of parallel programming. For example, developing a sorting algorithm using parallel programming is a helpful to fully understand the benefit. Using a vector that contains a high number of items can be sorted quicked using parallel programming.

Programming Demo

After establishing these conceptual foundations, we introduce OpenMP as a practical tool for parallel programming in C++. However, this step presents its own challenges. Many students struggle with the syntax of compiler directives, particularly the meaning and role of `#pragma`. Rather than treating it as a purely syntactic feature, we frame `#pragma omp` as an instruction to the compiler that enables parallel execution of code regions. By tying this back to the earlier animations and activities, students can interpret OpenMP not as "magic," but as a mechanism for expressing the parallel ideas they already understand conceptually.

Parallel programming is a programming model that allows a computer to use multiple resources simultaneously to solve computational problems. When computers use all their resources to solve a problem or process information, they are more efficient at performing tasks. Parallel programs can divide complex problems down into smaller tasks and process these individual tasks simultaneously. By separating larger computational problems into smaller tasks and processing them at the same time, parallel processing allows computers to run faster.

"Threads" are lightweight units of execution within a process. C++ offers "std::thread" for creating and managing threads, allowing different parts of a program to run concurrently.

Shared Memory: "Threads" within a single process can access a common memory space. This model is often used with libraries like OpenMP, which provides directives to parallelize code sections and loops.

Message Passing: Processes communicate by sending and receiving messages.

This is typically used for distributed memory systems, where processes run on different machines, and libraries like MPI (Message Passing Interface) facilitate communication.

In the simplest sense, parallel computing is the simultaneous use of multiple compute resources to solve a computational problem:

• A problem is broken into discrete parts that can be solved concurrently • Each part is further broken down to a series of instructions • Instructions from each part execute simultaneously on different processors • An overall control/coordination mechanism is employed

The computational problem should be able to:

• Be broken apart into discrete pieces of work that can be solved simultaneously • Execute multiple program instructions at any moment in time • Be solved in less time with multiple computer resources than with a single computer resource

The computer resources are typically:

• A single computer with multiple processors/cores • An arbitrary number of such computers connected by a network

Merge Sort Algorithm

In this example algorithm, a 'vector' of 'int' has been created and will be passed to the 'mergeSort' function to be sorted in ascending order. Clearly it is passed by reference, so that it can be updated, and 'left' and 'right' will be the first and last elements of the vector in the initial call to the function.

This is a recursive function, and will continue to make the recursive calls until 'left' is not less than 'right'. If 'left $<$ right' then it will divide the vector into two vectors or equal size. Then the 'mergeSort' function is called recursively with each of these two vectors. After the recursive calls are finished then a function 'merge' is called to add parts of the vector into ascending order. Code for the 'mergeSort' function is as follows:

Listing 15.1: Serial recursive merge sort implementation.

```
void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

Then the 'merge' function can be written as follows:

Listing 15.2: Merge Sort without recurrsion

```
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
        for (int i = 0; i < n1; ++i)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; ++j)
        R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
    }
}
```

Listing 15.3: Parallel Merge Sort without recurrsion

```
This is the code for the parallel Merge Sort algorithm:


    void parallelMergeSort(vector<int>& arr, int left, int
                           right, int depth = 0) {
                if (left < right) {
                        int mid = left + (right - left) / 2;
                                // Parallelize the recursive calls at shallow
                                   depth
                        if (depth < 4) {
                #pragma omp parallel sections
                #pragma omp section
                                parallelMergeSort(arr, left, mid, depth + 1);
                #pragma omp section
                                parallelMergeSort(arr, mid + 1, right, depth +
                                     1);
                        }
                        else {
                                parallelMergeSort(arr, left, mid, depth + 1);
                                parallelMergeSort(arr, mid + 1, right, depth +
                                     1);
                        }
                        merge(arr, left, mid, right);
                }
    }
```

Then the following code is used to call the non-parallel function, and calculate the time taken:

And to call the parallel merge sort function, the following code can be used to determine the time taken:

One result when executing this program provided the following times for each of the two functions:

Regular Merge Sort Time: 0.0241186 second Parallel Merge Sort Time: 0.0177899 second

It can be seen that the 'parallel' algorithm is able to achieve the sorting process faster than the non-parallel algorithm.

Changes in the Course

- `Linked List examples`: Several different examples of Linked Lists might not be needed, so they will not be covered.
- `Speed of algorithms`: This is something important for several algorithms, so including ways to improve the time taken to complete an algorithm is something important. These algorithms will be 'sorting', and can be 'searching' or using a 'graph' to find the shortest path between two cities.

File Organization

You will find the following file structure:

- `cover.tex`: Your chapter title, authors, and affiliations.
- `content.tex`: Your abstract, PDC description, and main body text.
- `appendix.tex`: Supplementary materials, git link, and rubrics.
- `figures/`: All image files (.png, .jpg, .pdf).
- `references.bib`: Your BibTeX bibliography file.

Cover Page Configuration (`cover.tex`)

Each chapter begins with a title and author block. Please update the `cover.tex` file using the following template. Ensure the chapter title is updated in **both** the bracketed short title (used for the Table of Contents) and the main title.

Citations

Use standard `\cite{label}` commands. Ensure all corresponding entries are in your `references.bib` file.

Abstract

|Your abstract text here.|

|Required student background|

|List topics here with bloom classification (use K-knowledge, C-comprehension, or A-be able to apply)|

PDC Concept	Chapter Section				
	1.1	1.2	1.3	1.4	1.5
Topic1	K	C	A		
Topic2	K	C	A		
Topic3		C	A	A	

1. |Outcome 1|
2. |Outcome 2|

|Your local context about institution, department, mix and levels of students, class size, language employed, etc.|

15.3 Introduction

|Motivation and background for the chapter.|

15.4 How CS1 was changed and PDC topics integrated

|Explain how the existing course (e.g., CS1) was modified.|

15.4.1 Integrated Syllabi (Pre and Post)

|Provide high level syllabi highlighting changes.|

15.4.2 Highlights

|What remained the same, and what specifically needed to be changed or deleted?|

15.4.3 Challenges

|Discuss logistical, pedagogical, or technical difficulties.|

15.5 New Unplugged Activities

15.5.1 Activity Name (e.g., Flag Maker)

Problem Description, Prerequisites, and Learning Outcomes

Implementation Details

- **Logistics (Material required for the activity, etc.)**
- **How to Conduct**
- **Instructional materiel (Lectures, Videos, Assignments, etc.)**

Experience and Teaching Tips

Data and Analysis

15.6 New Plugged-in Activities

15.6.1 Activity Name (e.g., Parallel Search)

Problem Description, Prerequisites, and Learning Outcomes

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)**
- **How-To Guides**

Experience and Teaching Tips

Data and Analysis

15.7 Course-wide Pre and Post Course Surveys

|Data collection and analysis of student feedback/learning.|

15.8 Other Activities and Ideas

|Alternative ways to apply these concepts, etc..|

15.9 Conclusion

Table 15.1: Comparison between serial and OpenMP parallel merge sort implementations.

Step	Serial Merge Sort	Parallel Merge Sort
Data Preparation	`vector<int> data1;` `readFile(data1);`	`vector<int> data2 = data1;`

Start Timer	`auto start =`
	`chrono::high_resolution_clock::now()`
	`;`
		`start =`
		`chrono::high_resolution_clock::now()`
		`;`
		`omp_set_num_threads(5);`

Sorting	`mergeSort(data1, 0,`
	`data1.size()-1);`
		`#pragma omp parallel`
		`{`
		`parallelMergeSort(`
		`data2, 0,`
		`data2.size()-1);`
		`}`

Thread Information	N/A	`#pragma omp critical`
		`{`
		`cout << "Threads: "`
		`<< omp_get_num_threads()`
		`<< endl;`
		`}`

End Timer	`auto end =`
	`chrono::high_resolution_clock::now()`
	`;`
		`end =`
		`chrono::high_resolution_clock::now()`
		`;`

Execution Time	`cout << "Regular Merge Sort Time:"`
	`<< chrono::duration<double>(`
	`end-start).count()`
	`<< " seconds\n";`
		`cout \<\< "Parallel Merge Sort Time:`
		`"`
		`<< chrono::duration\<double\>(`
		`end-start).count()`
		`<< " seconds\n";`

Table 15.2: Potential speedup for search algorithms.

Algorithm	Sequential	Parallelizable
Linear Search	0%	100%

Bibliography

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

`https://github.com/...`

Detailed Pre and Post Syllabus

Introduction to Programming Introduced the concept of parallel computing through unplugged activities and real-world examples of concurrency. (concepts only)

Loops and Iteration Discussed independent iterations, data parallelism, and dividing repetitive tasks among multiple workers. (concepts, programming demo)

Arrays, Vectors, and Strings Demonstrated data decomposition and partitioning of large datasets into smaller independent sections. (concepts, programming demo)

Searching and Counting Problems Compared sequential and parallel approaches using classroom activities and simple performance measurements. (concepts, programming demo)

Grid-Based Simulations Used zombie spread and image-processing style activities to illustrate parallel updates on large datasets. (concepts, programming demo, programming assignments)

Performance Measurement Introduced basic timing experiments, speedup, and efficiency using small C++ examples. (in class demo)

Organization of Material

|Explain the folder structure of your Github repository.|

Survey and Other Anonymized Data and Analysis

|Include student feedback, learning data, or implementation analysis.|